ASP.NET Core IHProcess Hosting

Back to ASP.NET Core Tutorials for Beginners and Professionals

ASP.NET Core IHProcess Hosting Model

In this article, I will discuss the **ASP.NET Core IHProcess Hosting Model** with Examples. Please read our previous article discussing the ASP.NET Core Startup class with an example.

What is the Hosting?

Hosting refers to the process that allows our web application to be accessible over the Internet. When we develop a web application, we need a server where our application's files, dependencies, and other resources can reside and be served to users who are accessing our application over the Internet.

What is a Web Server?

This is a physical or virtual machine that runs the server software, such as Apache, Nginx, or Microsoft's Internet Information Services (IIS). The server software is responsible for handling incoming HTTP requests from web clients (such as browsers) and delivering the requested web pages or data back to the clients.

What are Hosting Models in ASP.NET Core?

In ASP.NET Core, a hosting model defines how your application is hosted, configured, and served to handle incoming HTTP requests. The hosting model determines the process and environment in which your application runs and manages aspects such as performance, scalability, and deployment flexibility. There are two primary hosting models in ASP.NET Core:

- **In-Process Hosting**
- **Out-of-Process Hosting**

These hosting models determine how the ASP.NET Core application is hosted, which web servers are used, and how requests are processed. Understanding these hosting models is crucial for optimizing application performance and ensuring high scalability and reliability.

What is the IHProcess Hosting Model in ASP.NET Core?

In the IHProcess hosting model, the ASP.NET Core application runs inside the IIS worker process (w3wp.exe). This is the default hosting model for ASP.NET Core applications when deployed to IIS. The ASP.NET Core application runs directly within the IIS worker process, which means that IIS and the ASP.NET Core application share the same memory space and resources. This tight integration allows for efficient communication between IIS and the ASP.NET Core application.

The IHProcess Hosting Model

What is Kestrel Web Server?

Kestrel is a lightweight, cross-platform, open-source web server for ASP.NET Core applications. It is designed to be fast, secure, and scalable, making it a popular choice for hosting ASP.NET Core applications in various environments. Kestrel is used by default in the IHProcess hosting model and is also used in other hosting models such as Out-of-Process hosting.

How Does IHProcess Hosting Model Work in ASP.NET Core?

When a request arrives at the server, it is first received by the IIS web server. The request then goes through IIS's client handling or request processing pipeline. The ASP.NET Core module, which is a native module for IIS, intercepts the request and forwards it to the ASP.NET Core application running in the IIS worker process. The ASP.NET Core application processes the request and generates a response. The response then goes through the client handling or request processing pipeline in IIS and is sent back to the client.

Step 1: Client Sends Request to IIS

The client sends an HTTP request to the server, which is received by IIS. The request passes through IIS's client handling or request processing pipeline.

Step 2: IIS Forwards Request to ASP.NET Core Module (ANCM)

IIS is the web server that receives the incoming requests from the client over the Internet. Now, it will check whether the requested is a static or dynamic request.

- If the application has static content, IIS will forward the request to the ASP.NET Core Module (ANCM).
- If the application has dynamic content, IIS will forward the request to the ASP.NET Core Module (ANCM).

Step 3: ASP.NET Core Module (ANCM)

The ASP.NET Core Module (ANCM) is a native module for IIS that manages the ASP.NET Core application. It is responsible for starting and stopping the ASP.NET Core application and forwarding HTTP requests to the application running in the IIS worker process. The ASP.NET Core Module (ANCM) performs the following tasks:

- ANCM checks if the ASP.NET Core application is already running.
- If the application has not started yet, ANCM will start it. To start the application, ANCM loads the .NET Core runtime and the application's dependencies into the IIS worker process. It then sends a request to the ASP.NET Core application's entry point to initiate the application's startup process.
- Once it forwards the request to the IIS HTTP Server.

Step 4: IIS HTTP Server Receives the Request from the ASP.NET Core Module

The primary responsibility of the ASP.NET Core Module (ANCM) is to start the ASP.NET Core application running in the IIS worker process and to forward requests to the application. Once it has started the application, it will stop running, and IIS will directly forward the Kestrel to the IIS HTTP Server.

Step 5: IIS HTTP Server

The component acts as the HTTP protocol handler. It deals with encoding related to the HTTP protocol, such as URL, HTTP headers, and HTTP payloads. The IIS HTTP Server then sends the request to the IIS HTTP Server.

- The IIS HTTP Server receives the request from the ASP.NET Core Module.
- The request is processed through the ASP.NET Core middleware pipeline. This may involve ASP.NET MVC, etc., serving HTTP requests, accessing databases, and other tasks.
- Kestrel the processed request to the Application Code running in ASP.NET Core. Kestrel interprets the URL, headers, and other HTTP data and prepares it for the application.

Step 6: Application Code (ASP.NET Core Application)

This is where the business logic and data processing happen. The ASP.NET Core application has all the required code, the controllers, services, and middleware to handle the user request. The request goes through the ASP.NET Core middleware pipeline. Here, the application may:

- Authenticate/Authorize (e.g., checking if the user is allowed to access a particular resource).
- Log information about the request for monitoring and debugging purposes.
- Any other custom middleware components can have added that logging or exception handling.

Finally, the Controller Action processes the request. The controller action executes the relevant business logic, interacts with the data layer, and prepares the response data. The response data is then sent back to the IIS HTTP Server.

Step 7: IIS HTTP Server

The component acts as the HTTP protocol handler. It deals with encoding related to the HTTP protocol, such as URL, HTTP headers, and HTTP payloads. The IIS HTTP Server then sends the response back to the IIS HTTP Server.

Step 8: IIS HTTP Server

After receiving the processed response from the ASP.NET Core application, the IIS HTTP Server sends it back to IIS. The response goes from the application code to the IIS HTTP Server and then directly to IIS. IIS then sends the response back to the client.

Step 9: IIS

IIS sends the response over the Internet to the client. The client receives the response and displays it to the user.

Note: The IHProcess Hosting Model makes everything faster by hosting the ASP.NET Core application run inside the same process as IIS, allowing for efficient communication and resource sharing. This integration ensures high performance.

Examples to Understand IHProcess Hosting Model in ASP.NET Core Application.

Let's create a new ASP.NET Core Web Application. An ASP.NET Core Research creates the following Program.cs file with the default minimal hosting model.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.Map("/", (Func<string>)(() => "Hello World"));
app.Run();
```

When executing an ASP.NET Core Web Application, the .NET Core Runtime loads the first library, named.dll. The library related to the entry point for the .NET Core Application is loaded. As you can see in the above code, the **Main()** method of the Program class uses the **WebApplicationBuilder** class.

What are The Tasks Performed by the WebApplicationBuilder Method in ASP.NET Core?

The WebApplicationBuilder initializes and sets up the Web Application's configuration and dependencies across various setting default psychological settings. Its aim of setting up the Web Host and configuring services. the CreateBuilder() method performs several key tasks. These tasks include the following:

- Set up Web Server (IIS, IIS Express, or Kestrel).
- Load configuration from appsettings.ison and environment variables.
- Logging setup (console and remote logging).
- Configuration Unless access the Dark Sun Company's Data.
- Dependency Injection Configuration (Registering services for dependency injection).

What exactly does the CreateBuilder() method do with all the Web Host?

When we create a new ASP.NET Core Web Application, the CreateBuilder() method begins to configure IIS, and Kestrel. It prepares the application's dependencies, such as configuration and logging, using the CreateDefaultBuilder() method. It configures and set up the Web Host. The CreateDefaultBuilder() method Configures the Web Host with default settings in mind for the application's environment. The key tasks include:

- **Kestrel Configuration:** Kestrel is configured as the default web server and is optimized for serving ASP.NET Core applications.
- **IIS Integration:** If the application is hosted on IIS, the Web Host integrates with IIS for hosting and managing requests.
- **Environment:** It sets configurations for various environments (e.g., development, staging, production) and URLs for app listening.
- **Hosting Model:** It sets configuration settings to use IIS Integration or Out-of-Process hosting models.

The hosting model page will show you how to set up the application's environment to meet its needs, of Process Hosting in ASP.NET Core Hosting. Let's at the Discuss the In-Process Hosting Model and how to set up Process Hosting Model.

How Do the Configuration Services Work in ASP.NET Core?

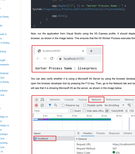
When we create a new ASP.NET Core Web Application using any project template in .NET 6 by default, the project is configured to use the In-Process hosting model. The configuration services are responsible for hosting IIS or IIS Express. The IHHosting Hosted authors better tasks the Out-Of-Process Hosting Model because the ASP.NET Core application runs within the IIS worker process, leveraging the performance and stability of IIS for managing HTTP requests and responses.

Now, let's discuss how to set up the Web Host using the In-Process Hosting model, which will discussed step:

Method 1: Using Project Properties GUI

To Manually configure the IHProcess Hosting in the ASP.NET Core Web Application, follow the only one simple setting, and the Project Properties GUI provides a user-friendly interface to adjust the hosting model. To set up the In-Process Hosting model using the Project Properties GUI, follow these steps: Right-click on the project in the Solution Explorer, and from the context menu, choose the option Properties.

Once you open the Properties Window, navigate to the Debug section. Under the Debug profiles, ensure that the **Enable IIS Express** option is selected. This ensure that IIS or IIS Express is used as the hosting environment for the application.



Once you open the Properties Window, navigate to the Debug section. Under the Debug profiles, ensure that the **Enable IIS Express** option is selected. This ensure that IIS or IIS Express is used as the hosting environment for the application.

Method 2: Specify the Hosting Model in IHProcess using CLI

To set up Kestrel with the In-Process hosting model using the project template, follow these steps: Create a new project, which will use the Project Properties window. Select the Debug tab from the window and click the **Open debug launch profiles UI** button to access the Project Properties.

Once you click on the Open debug launch profiles UI button, the Project Properties window will open. From the window, select IIS Express from the Debug profiles. Ensure that the Enable IIS Express option is selected. This ensures that the application will use IIS Express as the hosting environment. The Project Properties window also allows you to set the Application URL, environment variables, and other settings. These settings can be adjusted to suit your development needs. Once you have configured the hosting model and other settings, save the changes. After saving the changes, you can start the application and verify that it is running in the In-Process hosting model.

What Happens When We Set the Hosting Model as IHProcess?

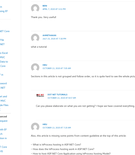
When configuring the hosting model as IHProcess and running the application using the IIS Express launch profile, the ASP.NET Core application runs directly within the **IIS Worker Process (w3wp.exe) or Internet Express or Development.** The application leverages IIS for hosting and managing HTTP requests, but the ASP.NET Core application runs within the IIS worker process, ensuring efficient communication and resource sharing.

Because the application runs in the same process as IIS, there is no additional overhead from network delays or context switching between processes. This setup can lead to better response times and throughput, making it ideal for high-performance scenarios environments.

The process name used to indicate the application runs in a **dotnet** in the **iis** of IIS. Similarly, in IIS Express, the process name will be **iisexpress**.

How Do We Configure Hosting Model in Run Application?

To run the IHProcess hosting Model in Visual Studio, we first need to set the launch profile to IIS Express, as shown in the Project Properties.



Then, click the **Open debug launch profiles UI** button, select IIS or IIS Express under the Project Properties tab and run the application. You will get the following output.

Now, let's discuss how Kestrel Hosting works for Business Applications in ASP.NET Core Hosting.

How Does Kestrel Hosting Model Work in ASP.NET Core?

Kestrel is a lightweight, well-endorsed version of Internet Information Services (IIS) designed for development in .net core. It is a cross-platform web server that is open-source and is optimized for serving ASP.NET Core applications. Kestrel is used by default in Windows and Linux applications hosting models before deploying them to production servers.

The Kestrel hosting model is ideal for the ease of deployment and ease of use in development, and in production. We need to set **IIS in production**. If our operating requirements, we will discuss deploying an ASP.NET Core Application on IIS.

Note: In order, Kestrel is the default hosting model for ASP.NET Core Applications. However, if you want to use Kestrel, it is recommended to implement it from the Reverse Proxy Server or Load Balancer. Then, we will discuss the Out-Of-Process Hosting Model.

In this article, we will discuss the Kestrel Server in ASP.NET Core Application with Examples. In this article, Kestrel in ASP.NET Core IHProcess Hosting with Examples. I hope you enjoy the IHProcess Hosting Model article.

Join Now 1,000,000+ Members
Sign up to get personalized notifications, save articles, and more!
Sign Up
Already a member? Sign In

16 thoughts on "ASP.NET Core IHProcess Hosting"

1. **Kamal** on **June 7, 2024 at 2:04 PM**

Its very unique and effective

Reply

2. **Kavindra Joshi** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

3. **Sudhir** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

4. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

5. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

6. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

7. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

8. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

9. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

10. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

11. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

12. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

13. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

14. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

15. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

16. **Ankit Kumar Singh** on **June 7, 2024 at 2:05 PM**

Thank you. Very useful

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

Post Comment

Home ASP.NET Core Tutorials ASP.NET Core Interview Questions ASP.NET Core Hosting Model ASP.NET Core Middleware ASP.NET Core Startup ASP.NET Core Hosting Model ASP.NET Core Tutorials ASP.NET Core Interview Questions and Answers Tutorials ASP.NET Core Tutorials ASP.NET Core Tutorials ASP.NET Core